Assignment 1B: Dynamic Programming

Sertan Erdogan (s5152283) & Hossein Kelisa (s5152135)
Group Number: 84

1 Introduction

Reinforcement Learning(RL) studies how an agent learns to make decisions by interacting with an environment. In this assignment, we investigate Dynamic Programming (DP) methods using the Windy GridWorld environment. Unlike Assignment 1A, which focused on exploration strategies, this assignment studies how optimal policies can be computed when the full environment model is known.

The Windy GridWorld consists of a 7×6 grid, where the agent starts at position $(0, 3)$ and must reach the goal at $(4, 3)$. The agent can move up, down, left, or right, but vertical wind in certain columns pushes the agent upward, making the task more challenging. In this assignment, we implemented a Value Iteration agent to compute the optimal policy and analyzed how parameters such as the discount factor γ influence the resulting behavior.

The wind affects the agent in specific columns of the grid. When the agent moves in these columns, it is pushed upward by a fixed number of cells depending on the wind strength of that column. This makes the transition dynamics more complex because the resulting next state depends not only on the chosen action but also on the environmental wind effect. As a result, the agent must learn policies that compensate for these vertical shifts while still reaching the goal efficiently.

2 Markov Decision Processes

The Windy GridWorld can be formulated as a Markov Decision Process (MDP), which is defined by the tuple (S, A, P, R, γ).

- S: the set of states. In this environment, each grid cell represents a state.
- A: the set of actions available to the agent (up, down, left, right).
- $P(s'|s, a)$: the transition probability of reaching state s' after taking action a in state s.
- $R(s, a, s')$: the reward received after a transition.
- γ: the discount factor determining how future rewards are weighted relative to immediate rewards.

The discount factor γ plays an important role. When $\gamma = 1$, future rewards are valued equally to immediate rewards. In environments where each step has zero reward, the agent may not strongly prefer shorter paths to the goal. By using $\gamma < 1$, future rewards are discounted, encouraging the agent to reach the goal more quickly while also ensuring convergence of value estimates.

3 Dynamic Programming and Generalized Policy Iteration

Dynamic Programming is a class of algorithms used to solve MDPs when the transition and reward functions are known. Instead of learning through trial-and-error interactions, the agent can directly compute value estimates using the model of the environment.

Many reinforcement learning algorithms can be viewed as instances of **Generalized Policy Iteration (GPI)**, which consists of two interacting processes: policy evaluation and policy improvement. During policy evaluation, the value of the current policy is estimated, while policy improvement updates the policy by selecting actions that maximize these value estimates.

Dynamic Programming algorithms such as Policy Iteration and Value Iteration follow this principle. Policy Iteration performs full policy evaluation before updating the policy, while Value Iteration combines evaluation and improvement into a single update rule, making it computationally more efficient.

4 Bellman Optimality and Value Iteration

The core of Dynamic Programming is the Bellman optimality equation, which defines the optimal action-value function as

$$Q(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q(s', a') \right].$$

This equation states that the value of taking action a in state s equals the expected reward plus the discounted value of the best action in the next state.

In Value Iteration, we repeatedly update the Q-values using this equation for every state-action pair until the values converge. The optimal policy can then be obtained by selecting the action with the highest Q-value:

$$a^* = \arg\max_a Q(s, a).$$

Compared to Policy Iteration, this approach avoids maintaining an explicit policy during training and often converges faster.

5 Experiments and Results

After training the ValueIterationAgent, the learned Q-values and corresponding optimal actions can be visualized in the Windy GridWorld environment. Figure 1 shows the resulting policy when the wind behaves deterministically.

The arrows indicate the optimal action for each state, while higher Q-values appear closer to the goal. The learned policy guides the agent along a path that avoids unnecessary wind effects when possible.

In the deterministic setting, the wind always pushes the agent upward with a fixed strength. Because the transition dynamics are predictable, the agent learns a policy that carefully navigates around the strongest wind columns. In particular, the optimal path often compensates for wind by selecting actions that move slightly downward before entering windy areas. This demonstrates how the agent incorporates environmental dynamics directly into its value estimates.

To study stochastic transitions, we repeated the experiment with wind blowing only with probability 0.75. The resulting policy is shown in Figure 2.

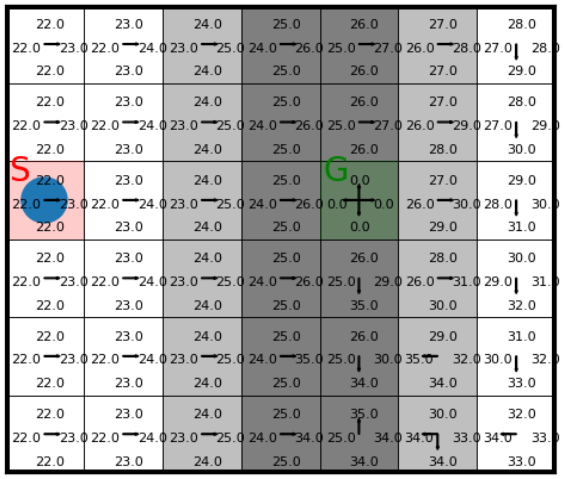

Figure 1: Optimal policy learned using Value Iteration in the deterministic Windy GridWorld environment.

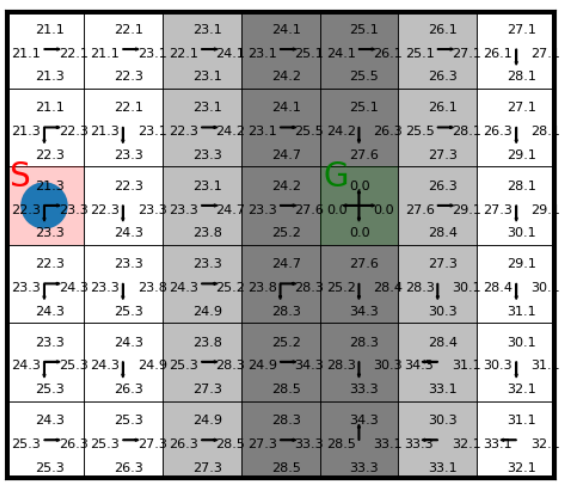

Figure 2: Optimal policy in the stochastic Windy GridWorld with wind probability 0.75**.**

In this case, the agent learns a slightly more cautious policy because transitions are uncertain. The Bellman update accounts for this uncertainty through the transition probabilities $P(s'|s, a)$, which allows the agent to compute optimal policies even in stochastic environments.

Compared to the deterministic case, the stochastic wind introduces uncertainty into the transition model. The agent must therefore consider the expected outcomes of actions rather than relying on fixed transitions. As a result, the learned policy tends to be slightly more conservative and may avoid risky states where strong wind could push the agent further away from the goal. This illustrates how Value Iteration naturally handles stochastic environments through the expectation over transition probabilities in the Bellman equation.

6 Conclusion

In this assignment, we studied Dynamic Programming methods for solving Markov Decision Processes using the Windy GridWorld environment. By implementing Value Iteration, we observed how repeated Bellman updates allow an agent to compute optimal action values and derive an optimal policy.

The experiments demonstrated how the discount factor γ influences the agent's preference for shorter paths and how stochastic wind affects the learned policy. These results illustrate the strong connection between Dynamic Programming, Bellman equations, and Generalized Policy Iteration, which form the theoretical foundation of many modern reinforcement learning algorithms.

Dynamic Programming methods, such as Value Iteration, provide an important conceptual foundation for reinforcement learning, even though they require full knowledge of the environment model. Understanding these algorithms helps motivate later model-free methods that learn optimal policies directly from experience.